

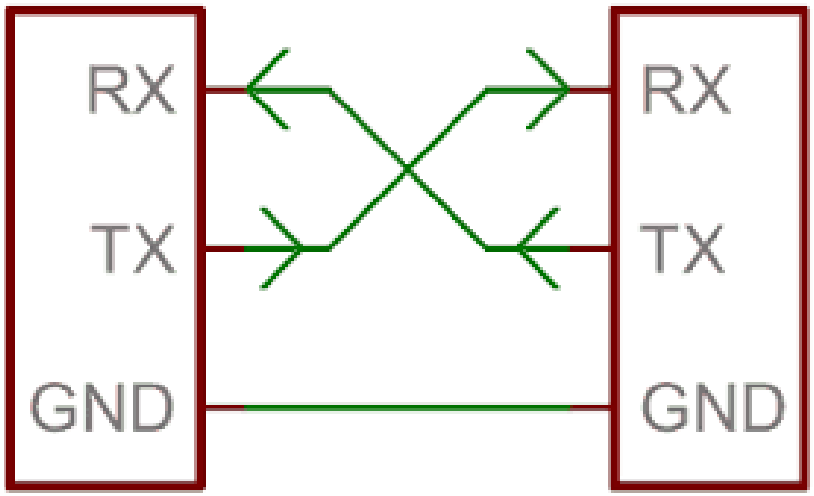
Proiectarea cu Microprocesoare
Curs 4

Andrei Voinescu

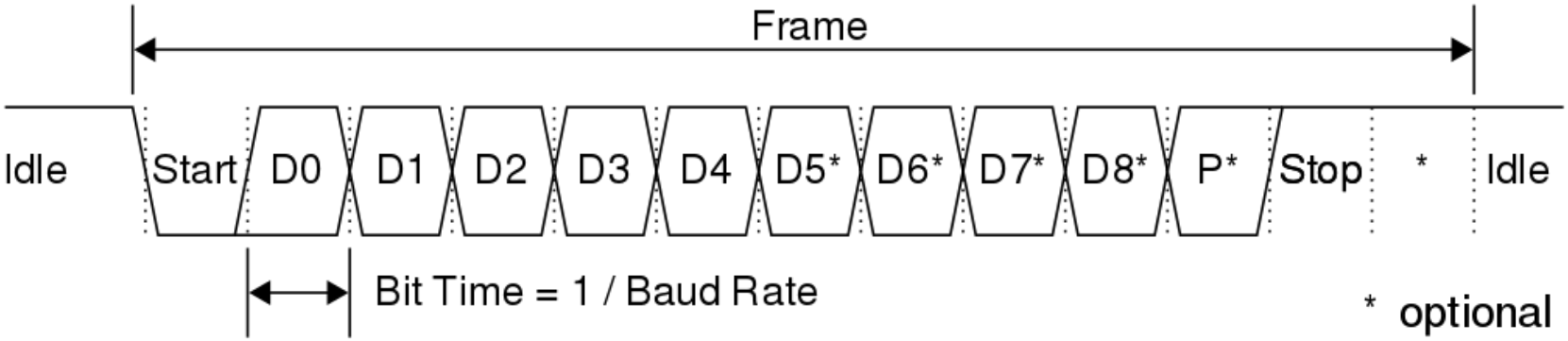
POLITEHNICA University of Bucharest

18 martie 2020

- Modalități de interconectare între dispozitive, sau între chip-uri din cadrul aceluiași dispozitiv
- În contrast cu interfețele paralele, presupune înșiruirea mai multor biți pe aceeași linie de date
- Duplex/ Half-duplex / Full-duplex
  - Duplex se referă la posibilitatea de a avea comunicație în 2 sensuri pe 2 canale diferite.
  - Comunicația Full-duplex permite comunicației să aibă loc simultan în ambele sensuri
  - Comunicația Half-duplex nu permite comunicația într-un singur sens la un moment dat



- Conexiuni doar între două dispozitive
- Semnalele sunt legate în cruce



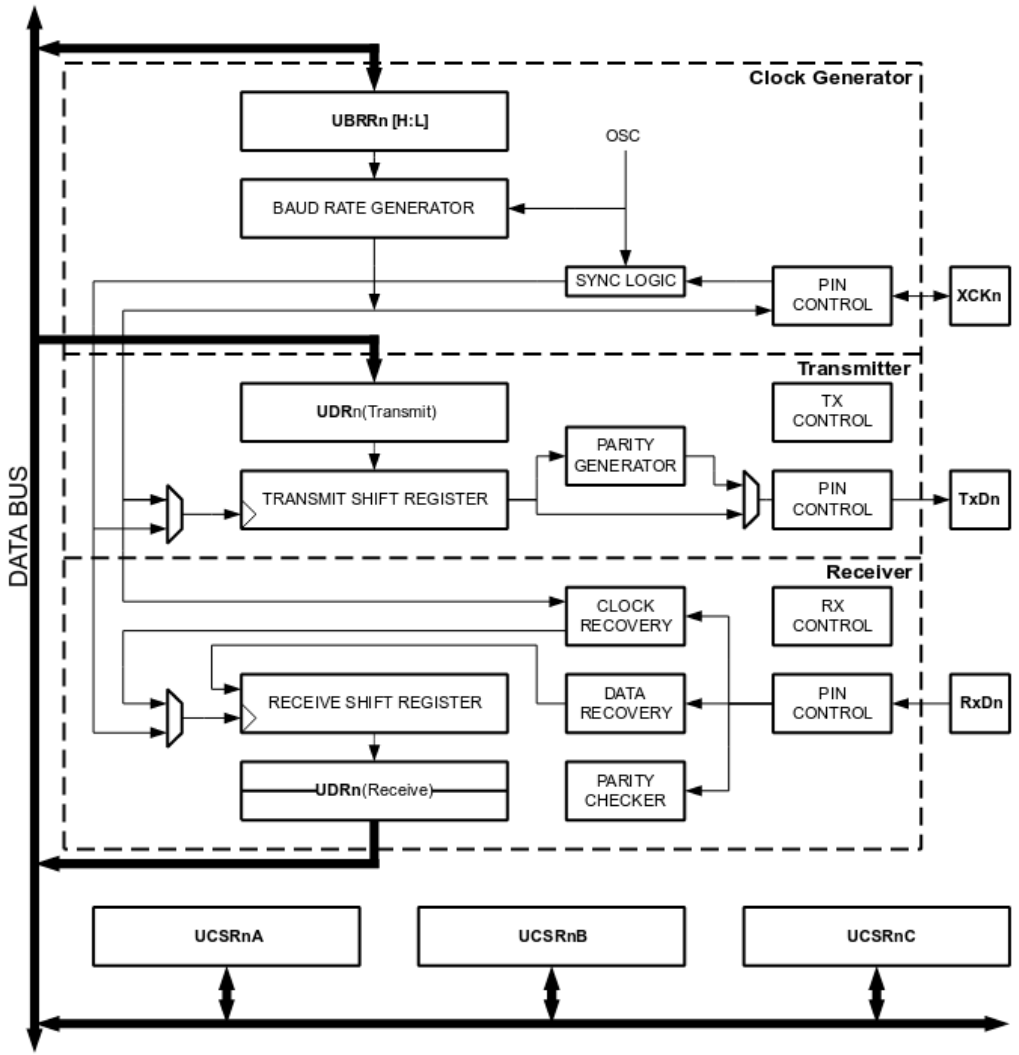
- Start-ul este dat de o tranziție a liniei între 1 și 0
- Datele pot avea între 5 și 9 biți
- Datele pot fi verificate cu un bit de paritate
- Sunt 1 sau 2 biți de STOP

[0]Sursă imagine: Wikipedia Commons

- Universal Synchronous/Asynchronous Receiver Transmitter
- Comunicație full-duplex
- UART - Protocol cu ceas **implicit**
  - Ambii parteneri în comunicație (cele 2 dispozitive) folosesc același ceas în transmisie/recepție
  - Similar, toți parametrii comunicației trebuie să fie identici
    - Frame size
    - Stop bits
    - Parity bit
- USART - ceas **explicit** - `XCK`
  - Generat de master
  - Slave-ul trebuie să se sincronizeze

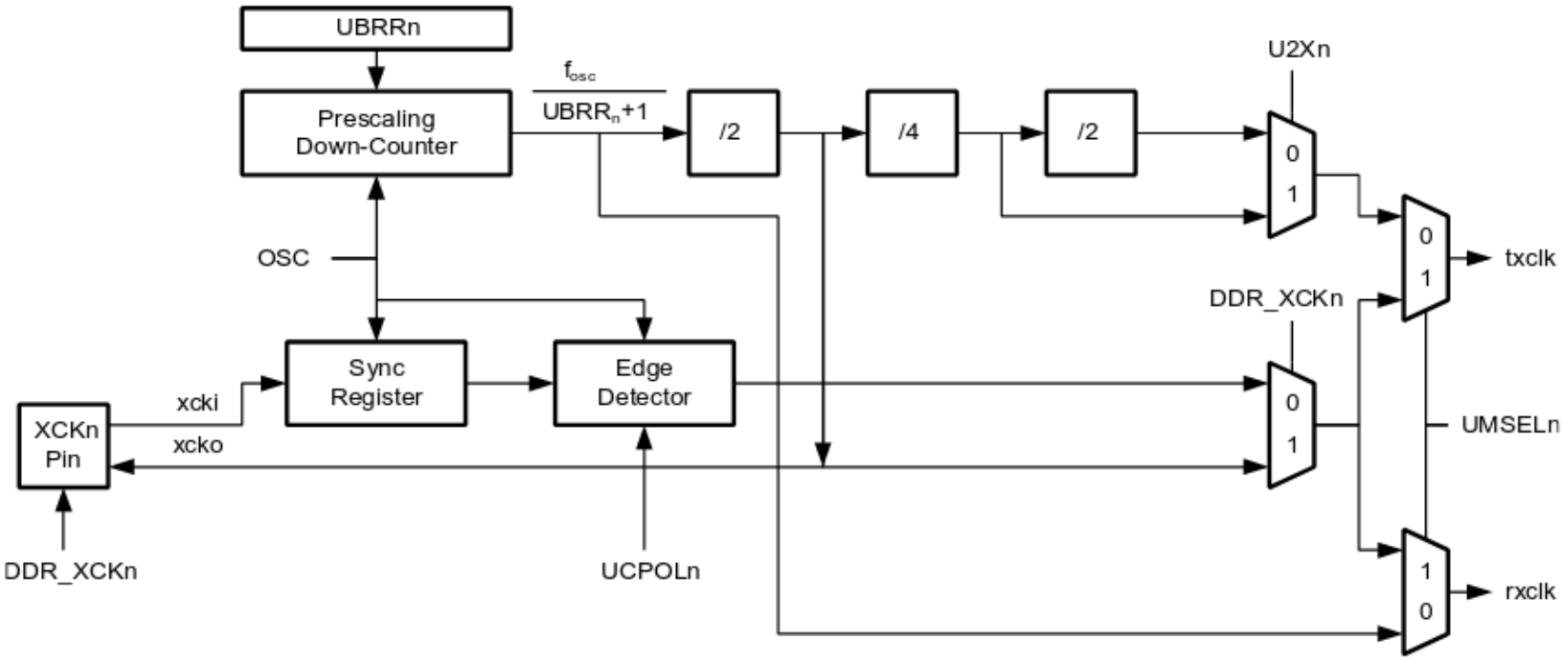
UART pe ATMega324a - arhitectură

Figure 21-1. USART Block Diagram



[0]Sursă imagine: Datasheet ATMega324a

- Ce trebuie să controlăm
  - Structura unui cadru (frame)
  - Viteza de tx/rx
- Ce trebuie să aflăm
  - Flow date în/din buffer
  - Verificări paritate



- Datorită sincronizării între domenii de timp

$$f_{XCK} < \frac{f_{OSC}}{4}$$

[0]Sursă imagine: Datasheet ATMega324a

1. Receptorul caută START
   - Detectează o tranziție 1→0
   - Citiri repetate (3) în mijlocul bitului
   - 3 x 0 → START valid
2. Citește fiecare bit din frame
3. Citește STOP bit ca 1 (altfel *Frame Error*)

- Surse de erori:
  - Frecvențele de tx/rx nu sunt egale
  - Frecvențe instabile
  - Glitch-uri pe linia de transmisie (START fals)
  - All of the above
- Frame Error - bit în registru de status
- Parity Error - bit în registru de status

- Format cadru - identic
- Două câmpuri de biți de configurare în plus
  - UMSEL - mode select
  - UCPOL - clock polarity
- O linie de semnal în plus
- Frecvențe de comunicație mai mari
- Sursele de eroare mult diminuate

- UBRR0
  - Registru prescaler pentru generarea ceasului de transmisie
  - Prescaler-ul de la f_{CPU} este $16 \cdot UBRR0$
  - `UBRR0` poate fi calculat sau poate fi extras din tabele existente în *datasheet*
  - $$UBRR = \frac{f_{CPU}}{16 \cdot baud} - 1$$
  - putem dubla viteza cu același `UBRR`
  - $$UBRR_{U2X} = \frac{f_{CPU}}{8 \cdot baud} - 1$$
  - $$Error_{[\%]} = \left(\frac{Baudrate_{closest}}{Baudrate} - 1\right) \cdot \%$$

- UCSRnA (status)
  - `UDRE` - 1 când buffer-ul de ieșire este liber
  - `RXC` - 1 când am primit un frame nou
  - `U2X` - 1 dacă micșorăm prescalerul la 8
  - `FE` - 1 dacă a fost Frame Error
  - `UPE` - 1 dacă a fost Parity Error
- UCSRnB (biți enable)
  - `TXEN` - Activarea transmițătorului
  - `RXEN` - Activarea receptorului
  - `RXCIE`, `TXCIE`, `UDRIE` - Activarea întreruperilor pentru RX complete, TX commplete, buffer empty
- UCSRnC (parametri frame)
  - `USBS` - numărul de biți de stop
  - `UCSZ` - dimensiunea unui frame
  - `UPM` - configurarea bitului de paritate

- inițializare USART0 9600,8N1

```
UCSR0C |= (1 << TXEN0) | (1 << RXEN0);
UBRR0 = 103;
```

- inițializare USART0 115200,8E2

```
UCSR0B |= (1 << TXEN0) | (1 << RXEN0);
UCSR0C |= (1 << UPM01) | (1 << USBS0) | (3 << UCSZ00);
UBRR0 = 8;
```

- inițializare USART1 2000000 (2Mbaud), 8O2

```
UCSR1A |= (1 << U2X1);
UCSR1B |= (1 << TXEN1) | (1 << RXEN1);
UCSR1C |= (1 << UPM10) | (1 << USBS1) | (3 << UCSZ10);
UBRR1 = 0;
```

- Receive simplu

```
loop_until_bit_is_set(UCSR0A, RXC0);
char c = UDR0;
```

  - Busy-waiting

- Receive pe întrerupere

```
volatile char c;
ISR(USART0_RX_vect)
{
    c = UDR0;
}
```

  - Fără busy-waiting, dar se pierd date dacă nu le citește nimeni din variabila shared



UART snippets

```
#define BUFSIZE     (100)

volatile uint8_t rx_buffer[BUFSIZE];
volatile uint8_t read_index = 0, write_index = 0;

ISR(USART0_RX_vect)
{
    uint8_t next_write = (write_index + 1) % BUFSIZE;

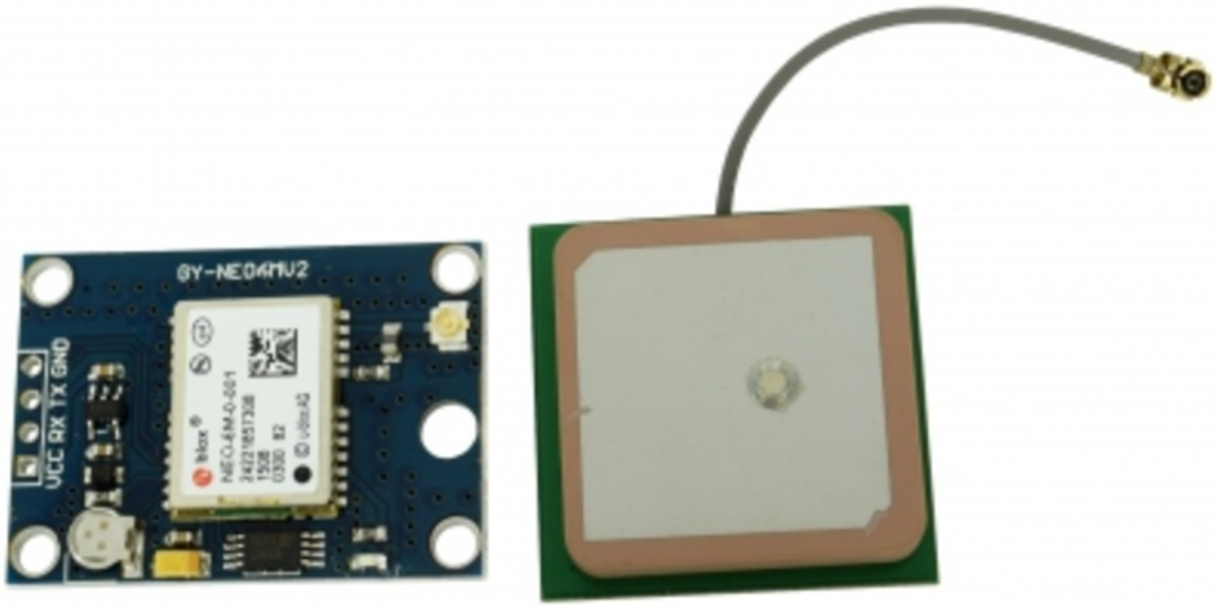
    if (next_write != read_index) /* daca nu e plin bufferul */
    {
        rx_buffer[write_index] = UDR0;
        write_index = next_write;
    }
}

int main()
{
    init_stuff();
    sei();

    while(1)
    {
        while(read_index != write_index) /* cata vreme avem elemente */
        {
            LCD_putChar(rx_buffer[read_index]);
            read_index = (read_index + 1) % BUFSIZE;
        }
    }
    return 0;
}
```

- Necesitatea unui buffer
  - Dacă folosim busy-waiting nu avem multă nevoie, dar avem multe alte dezavantaje
  - Bufferul din periferic reține un singur frame (5-9 biți)
  - Un buffer software ne va garanta că nu pierdem caractere

UART exemplu - GPS



- Modulele GSM/GPS de multe ori folosesc UART
- De ce?

CFG_COM1	CFG_COM0	Protocol	Messages	UARTBaud rate	USB power
1	**1**	**NMEA**	**GSV, RMC, GSA, GGA, GLL, VTG, TXT**	**9600**	**BUS Powered**
1	0	NMEA	GSV, RMC, GSA, GGA, GLL, VTG, TXT	38400	Self Powered
0	1	NMEA	GSV[14], RMC, GSA, GGA, VTG, TXT	4800	BUS Powered
0	0	UBX	NAV-SOL, NAV-STATUS, NAV-SVINFO, NAV-CLOCK, INF, MON-EXCEPT, AID-ALPSERV	57600	BUS Powered

- Configurația pinilor dictează protocoalele
- Pentru 1 și 1 avem UART 9600 baud cu formatul GGA

Table 1-3 contains the values for the following example:

$GPGGA,161229.487,3723.2475,N,12158.3416,W,1,07,1.0,9.0,M, , , ,0000*18

Table 1-3 GGA Data Format

Name	Example	Units	Description
Message ID	$GPGGA		GGA protocol header
UTC Time	161229.487		hhmmss.sss
Latitude	3723.2475		ddmm.mmmm
N/S Indicator	N		N=north or S=south
Longitude	12158.3416		dddmm.mmmm
E/W Indicator	W		E=east or W=west
Position Fix Indicator	1		See Table 1-4
Satellites Used	07		Range 0 to 12
HDOP	1.0		Horizontal Dilution of Precision
MSL Altitude	9.0	meters	
Units	M	meters	
Geoid Separation		meters	
Units	M	meters	
Age of Diff. Corr.		second	Null fields when DGPS is not used
Diff. Ref. Station ID	0000		
Checksum	*18		
<CR> <LF>			End of message termination



Checklist

- Conectăm modulul la ATMega324 (TX la RX, RX la TX)
- Masa și alimentarea, pinii de configurare pentru UART 9600 baud
- Configurăm seriala UART 0/1 pentru comunicație la 9600 baud
- Așteptăm frameuri pe serială
- Citim frameurile într-un buffer, parsăm atunci când am primit "tot"

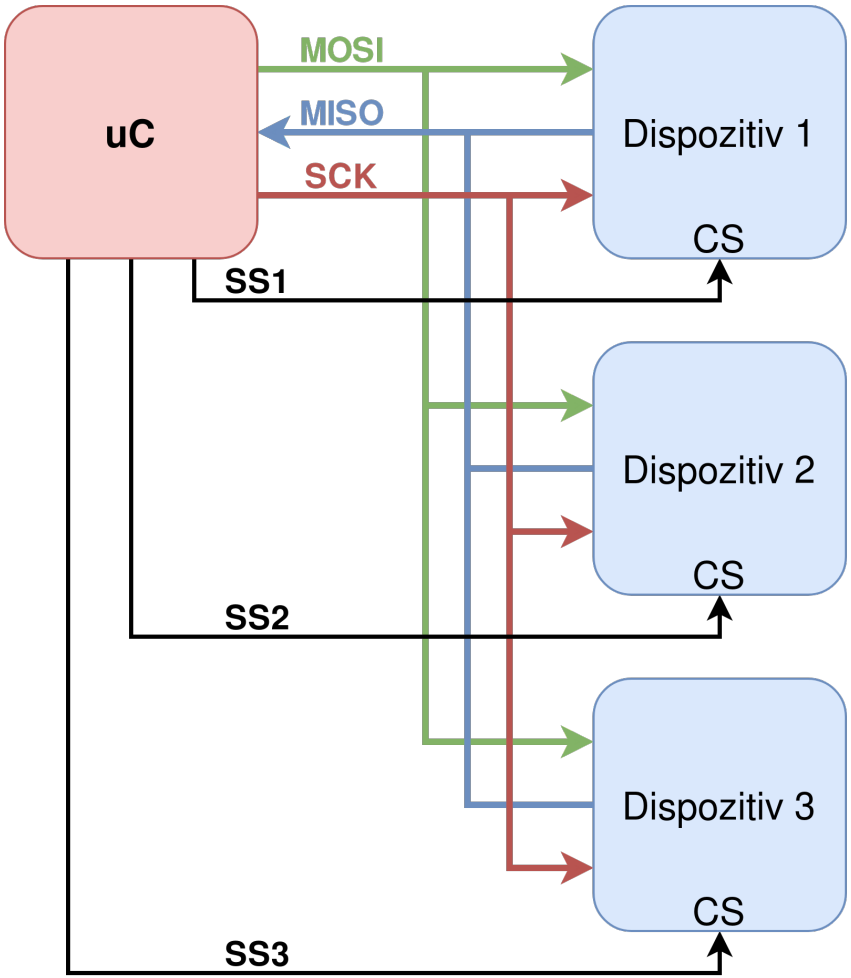
- Cum se rezolvă?
- Cum se face inițializarea?
- Cum se face recepționarea datelor?
- Unde/Când sunt parsate datele?
- Ce facem cu rezultatul procesării datelor?



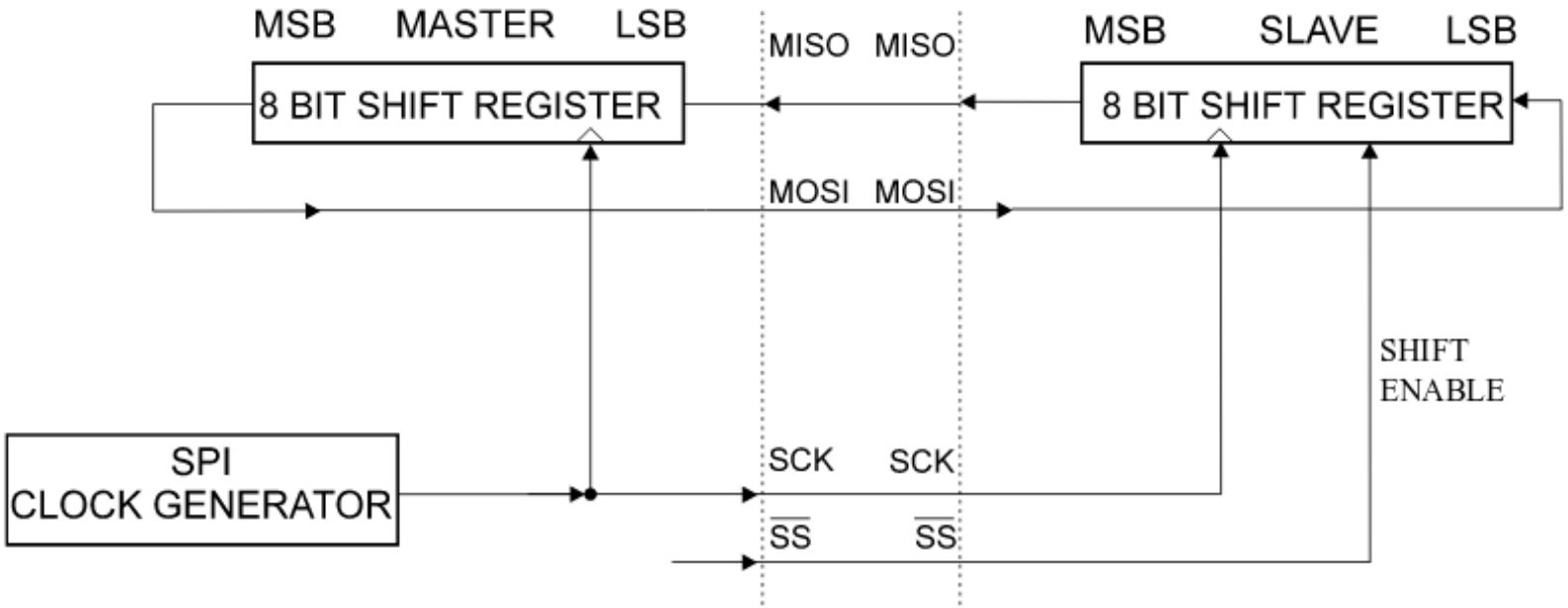
UART - concluzii

- Ceas implicit
- Transmițător independent de receptor
- Susceptibil la erori
- Error correction
- Mod sincron

- Serial Peripheral Interface
- Ce presupune
  - Comunicație sincronă 1:N
- Când este folosită
  - Dispozitive periferice externe cu bandwidth mare



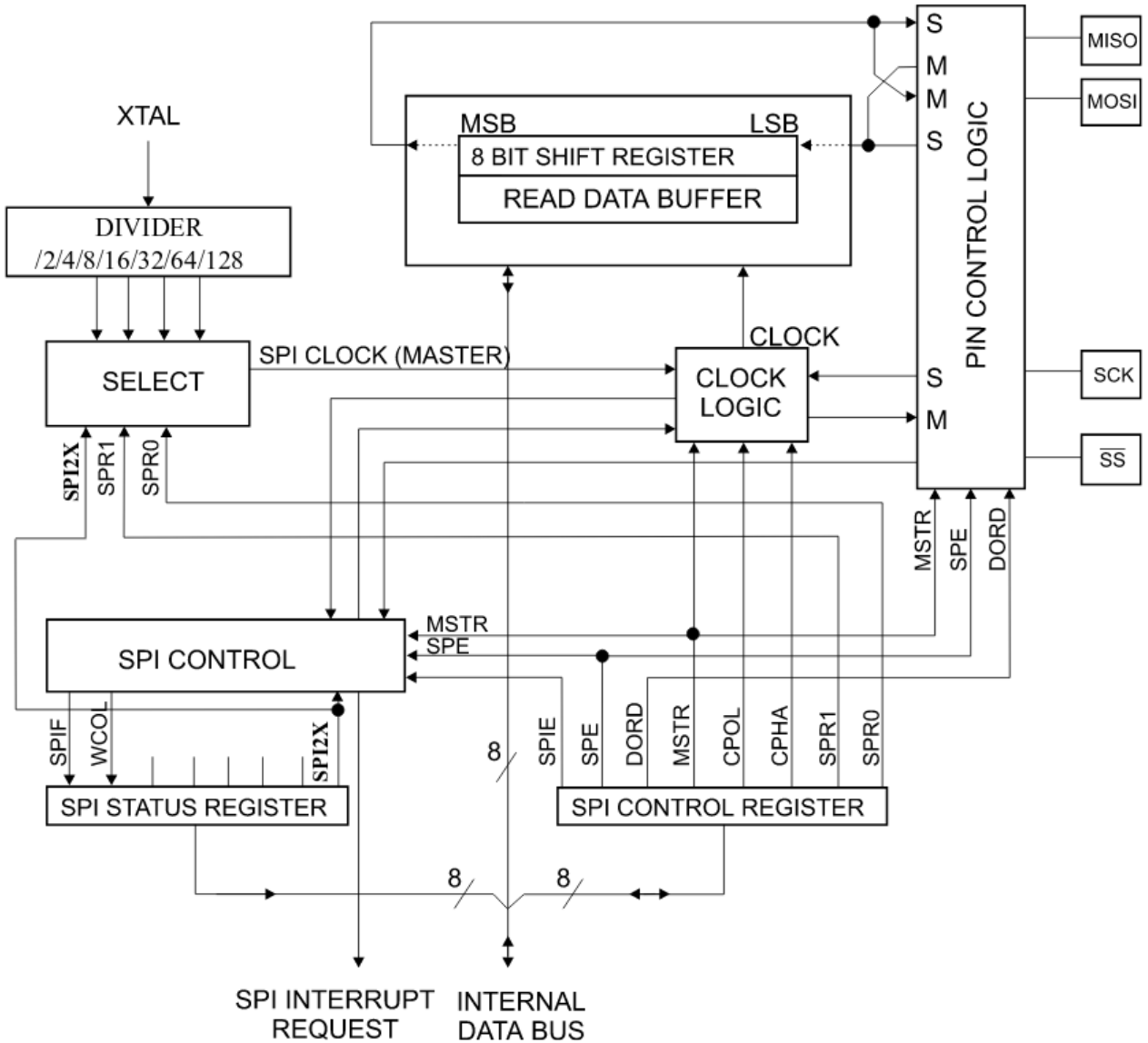
- 4 linii de date pt device
- 3 linii comune
- câte o linie de select pt fiecare device
- linii unidirecționale
- `MOSI` - Master Output Slave Input
- `MISO` - Master Input Slave Output
- `SCK` - Serial Clock
- `SS/CS` - Slave/Chip Select



- MSB de la master devine LSB la Slave (configurabil invers)
- Se formează practic un Rotate Left pe 16 biți
- Cu 8 biți pe Master și 8 biți pe Slave

[0]Sursă imagine: Datasheet ATMega324a

SPI pe ATMega324a - arhitectură



[0]Sursă imagine: Datasheet ATMega324a

- Ce trebuie să controlăm
  - Modul Master/Slave
  - Viteza de transmisie/polaritate
  - Select Chip - se face manual cu GPIO (la fiecare tranzacție)
- Ce trebuie să aflăm
  - Dacă o transmisie s-a terminat

- SPCRn - registru de control
  - SPIE - 1 dacă întreruperea SPIF este activă
  - SPE - 1 dacă SPI este pornit
  - DORD - 1 dacă LSB se trimite primul
  - MSTR - 1 pentru master
  - CPOL:CPHA - Clock polarity & phase
  - SPR1:0 - SPI Data Rate
- SPSRn - registru de status
  - SPIF - 1 dacă un transfer e complet
  - SPI2X - dublează data rate-ul din SPR1:0
- SPDRn - buffer de date

- Comunicație pe SPI full-duplex

```
uint8_t spi_xchg(uint8_t date) {
  SPDR = date;
  loop_until_bit_is_set(SPSR, SPIF);
  return SPDR;
}
```

- Citire pe SPI

```
uint8_t spi_read() {
  return spi_xchg(0xFF);
}
```

- Scriere pe SPI

```
spi_write(uint8_t in) {
  spi_xchg(in);
}
```

- Full-duplex sau half-duplex?
  - Documentație periferic (ex: Accelerometru ADXL362)

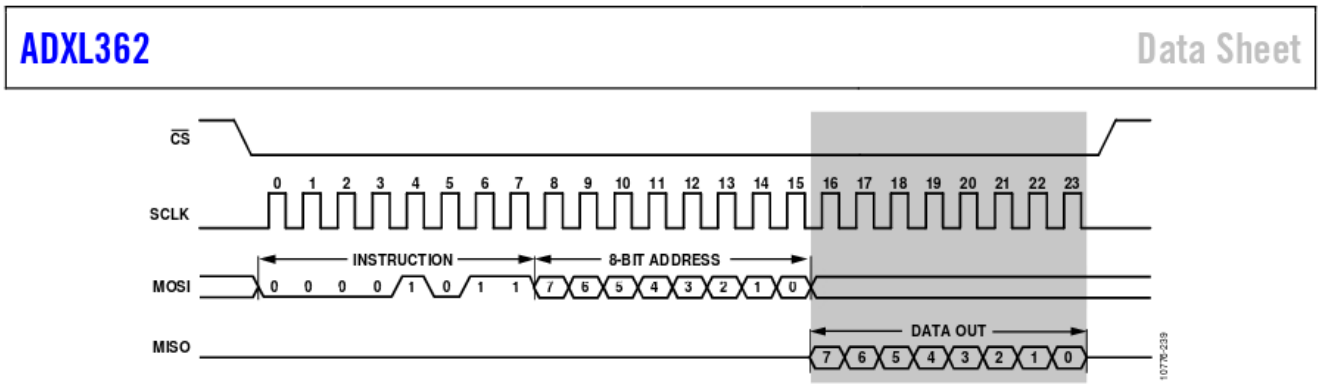


Figure 36. Register Read

  - Vrem să citim de la adresa 0x08 din memoria perifericului (XDATA)

```
spi_write(0x0B); //comanda register read
uint8_t valoare = spi_xchg(0x08); // gresit!
```

  - Dispozitivul nu are cum să răspundă instant la adresă

```
spi_write(0x0B); // comanda register read
spi_write(0x08); // adresa de citit
uint8_t valoare = spi_read(); // corect!
```

[0]Sursă imagine: Datasheet ADXL362

- Senzori (accelerometre, altimetre, giroscop etc.)
- Display-uri
- Memorii (Flash, carduri SD/MMC)
- Este extins pentru bandwidth
  - Dual, Quad, Octo - SPI
  - 2, 4, 8 liniile de date bidirecționale
  - Inițial pe Single SPI



SPI - concluzii

- Ceas explicit - fără erori
- Viteză de transmisie mare (f. mare pentru QSPI OSPI)
- Diferențe mici față de UART sincron
- Necesită multe linii pentru mai multe dispozitive (n+3)